

Objective

To implement and demonstrate Multilevel Inheritance and Interface Implementation in Java, where Puppy inherits from Dog, Dog inherits from Animal and implements the Pet interface.

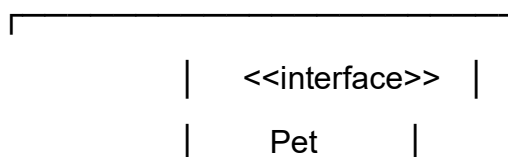
Steps for Execution

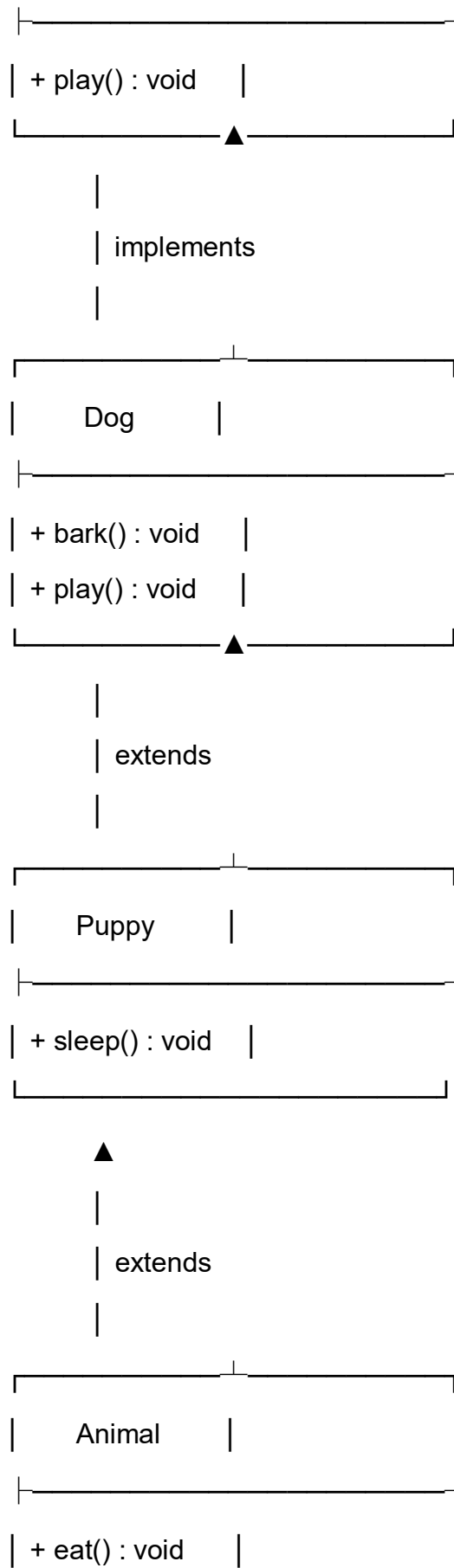
1. Open a Java IDE such as Eclipse, IntelliJ IDEA, or VS Code.
2. Create a Java project and package named javacore.
3. Create a file named Main.java and enter the given code.
4. Define the Pet, Animal, Dog, and Puppy classes.
5. Compile the program.
6. Run the Main class.
7. Observe the output.

Algorithm

1. Start the program.
2. Define the Pet interface with play().
3. Define Animal with the eat() method.
4. Create Dog by extending Animal and implementing Pet.
5. Create Puppy by extending Dog.
6. Create a Puppy object.
7. Call eat(), bark(), play(), and sleep().
8. Display the output.
9. Stop the program.

UML CLASS DIAGRAM:





Relationships:

- Dog extends Animal
- Dog implements Pet
- Puppy extends Dog

CODE

```
package javacore;
```

```
interface Pet {  
    void play();  
}
```

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
class Dog extends Animal implements Pet {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
  
    public void play() {  
        System.out.println("Dog plays");  
    }  
}
```

```
class Puppy extends Dog {
```

```
void sleep() {  
    System.out.println("Puppy sleeps");  
}  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        Puppy p = new Puppy();  
  
        p.eat();  
        p.bark();  
        p.play();  
        p.sleep();  
    }  
}
```

OUTPUT:

Animal eats

Dog barks

Dog plays

Puppy sleeps

RESULT:

The program successfully demonstrates Multilevel Inheritance with Interface Implementation in Java.

Here, Puppy inherits from Dog, and Dog inherits from Animal while implementing the Pet interface. Therefore, the Puppy object can access methods from all three levels: eat(), bark(), play(), and sleep().